IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 31 October 2022, accepted 5 December 2022, date of publication 15 December 2022,
date of current version 20 December 2022.

Digital Object Identifier 10.1109/ACCESS.2022.3229370

RESEARCH ARTICLE

Toward Smart-Building Digital Twins: BIM and IoT Data Integration

DAGIMAWI D. ENEYEW[1], (Graduate Student Member, IEEE), MIRIAM A. M. CAPRETZ[1], (Senior Member, IEEE), AND GIRMA T. BITSUAMLAK[2]
[1]Department of Electrical and Computer Engineering, Western University, London, ON N6A 5B9, Canada
[2]Department of Civil and Environmental Engineering, Western University, London, ON N6A 3K7, Canada

Corresponding author: Dagimawi D. Eneyew (deneyew@uwo.ca)

This work was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC) Discovery through Western University under Grant RGPIN-2021-04161 and Grant RGPIN-2018-05454.

ABSTRACT Smart-building digital twins aim to virtually replicate the static and dynamic building characteristics through real-time connectivity between the virtual and physical counterparts. The virtual replica of the building can then be leveraged to monitor the current state, predict the future state, and take proactive measures to ensure optimal operation. Despite its potential, smart-building digital twin research is at an early stage compared to manufacturing and aerospace fields. One of the major impediments to adopting digital twin technology in smart buildings is the lack of interoperability, primarily between Building Information Modeling (BIM) and Internet of Things (IoT) data sources. Consequently, this paper presents a novel multi-layer digital twin architecture for smart buildings called BIM-IoT Data Integration (BIM-IoTDI) to enable semantic interoperability among smart-building digital twin applications. A detailed framework is presented based on the newly developed architecture, introducing an ontology-based query mediation method that provides integrated data access. An experimental evaluation model is developed to characterize the feasibility of the BIM-IoTDI architecture and framework. Furthermore, the performance of the new query mediation method is evaluated and compared to an existing BIM-IoT data integration approach. According to the evaluation results, the BIM-IoTDI architecture and framework are better suited to supporting the envisioned smart-building digital twin capabilities.

INDEX TERMS BIM, data integration, data interoperability, digital twin, IoT, ontology, semantic interoperability, smart building.

I. INTRODUCTION

Buildings constitute a significant portion of the built environment contributing to nearly 40% of the total global energy use and one-third of global greenhouse gas emissions [1]. Despite efforts to reduce building energy consumption and emissions, the trend is still rising [2]. Consequently, there is an urgent need to improve energy efficiency and reduce building emissions in order to achieve sustainable environment goals [3].

Buildings are complex cyber-physical systems with multiple interacting subsystems to ensure safe, efficient, and seamless functioning. Due to the complexity of buildings and the necessity for efficient and secure operations, modern technologies such as the Internet of Things (IoT) and Artificial Intelligence (AI) are being applied, giving rise to smart buildings [4]. With their embedded technologies, smart buildings allow for a more sustainable, efficient, flexible, and interactive operation [5], [6].

In recent years, the concept of Industry 4.0, also known as *"The fourth industrial revolution"* has been introduced, in which real-time sensor data, cyber-physical connectivity, and autonomous control are utilized to boost productivity [7], [8], [9]. Respective initiatives such as Construction 4.0 [10] and Building 4.0 [11] adopted a similar concept to transform processes and methods in the Architecture Engineering and

The associate editor coordinating the review of this manuscript and approving it for publication was Shaohua Wan.

Construction (AEC) industry. One of the key enablers of Industry 4.0, Construction 4.0, and Building 4.0 is digital twin technology.

Digital twin (DT) is an emerging concept in a number of different fields. Digital twin, as defined by Liu et al. [12], is a living model of a physical asset or system that continually evolves alongside its physical counterpart based on continuous real-time data collection and can predict its future state. Digital twins have three main components: the physical entity, virtual entity, and bidirectional connectivity between the physical and virtual entities [13], [14], [15]. In the context of smart buildings, the primary objective of digital twins is to improve building functionality and efficiency through real-time monitoring, optimization, autonomous control [14], [16], [17], and interactive visualization [15]. In recent years, the application of the digital twin concept to smart buildings has gained increasing attention. Despite growing interest in digital twins, research in the Architectural Engineering and Construction (AEC) domain is still in its early stage in comparison to the manufacturing and aerospace domains [18], [19].

Smart-building digital twins aim to virtually replicate the static and dynamic aspects of a smart building throughout its entire life cycle phases (i.e., design, construction, operation, and maintenance) [20], [21]. Different types of data are generated for various purposes at each phase of a building's life cycle. Building Information Modeling (BIM) data, for instance, provide high-fidelity geometric, semantic, and spatial information regarding building elements during the design, construction, and maintenance phases [22], [23]. On the other hand, Internet of things (IoT) technology provides dynamic and real-time operational data during the operation and maintenance phases of a smart building [22]. Due to the complementary nature of BIM and IoT data sources in capturing static and dynamic building information, their integration paves the way for the development of digital twins of smart buildings [16], [20], [24]. Consequently, integrating smart building data from independent, cross-domain, and heterogeneous data sources such as BIM and IoT is one of the most important research problems to solve [16], [19], [20]. These data sources typically remain siloed within their own environments, hindering data interoperability. However, to achieve various cognitive functions of smart-building digital twins, such as reasoning, learning, and decision making, interoperable static and dynamic building data are essential [14], [25]. In particular, semantic-level interoperability that ensures the description of data using explicit domain-specific terminology (i.e., ontology) [25], [26] is an ideal level of interoperability for heterogeneous and cross-domain data environments. Smart-building digital twins require semantic interoperability to provide a comprehensive solution for improving a building's efficiency and functions by leveraging all information related to a building, regardless of the data format used by heterogeneous building data sources. In addition, semantic interoperability enables smart-building digital twin applications to exchange data with unambiguous and shared meaning.

Solving the interoperability issues between static and dynamic building data, primarily from BIM and IoT data sources, has been the subject of extensive research. Some research has focused on integrating BIM and IoT data for digital twins of smart buildings, whereas others have examined the general BIM-IoT integration paradigm. For instance, digital twin architectures [17], [19], [27] and frameworks [14], [28], [29] for smart buildings have been presented to address the BIM and IoT data integration issue. However, the proposed architectures and frameworks lack in integrating BIM and IoT data to enable semantic interoperability among smart-building digital twin applications while maintaining data in their optimal environment. Besides, to the best of the authors' knowledge, the presented digital twin frameworks for smart buildings have yet to be validated in actual building data. Furthermore, the BIM-IoT integration methods introduced in the smart-building digital twin context and the BIM-IoT paradigm have limitations in supporting the envisioned digital twin functionalities. These limitations include reliance on vendor-specific Application Programming Interface (API), data exchange with no shared meaning, utilizing semantic data representation exclusively in BIM, and storing dynamic sensor data outside its more efficient environment.

In this paper, a multi-layer digital twin architecture and framework for smart buildings, named BIM-IoTDI, is presented in an effort to address the aforementioned limitations. The BIM-IoTDI architecture defines the high-level structure of smart-building digital twins in terms of layers and the main components required in each layer. The BIM-IoTDI framework, on the other hand, presents methods for fulfilling each layer's primary purpose, with a specific focus on solving data integration and interoperability challenges. The primary contributions of this study can be summarized as follows:

- A novel multi-layer digital twin architecture for smart buildings is developed to enable semantic interoperability among smart-building digital twin applications. In BIM-IoTDI architecture, static and dynamic building data sources, primarily BIM and IoT data, are represented by domain ontologies laying an essential foundation for semantic interoperability through the shared meaning of data.
- As a critical component of the BIM-IoTDI framework, an ontology-based method for integrating BIM and IoT data based on query mediation is developed. The BIM-IoTDI integration method provides integrated data access for smart-building digital twin applications while maintaining the IoT data in optimal time-series databases.
- An experimental evaluation model that characterizes the feasibility of the BIM-IoTDI architecture and framework is developed using actual building data.

The remainder of this paper is organized as follows. Section II examines the current state of smart-building digital twin and BIM-IoT data integration research. The BIM-IoTDI architecture for smart buildings is described in Section III. Section IV explains the BIM-IoTDI framework in detail. The BIM-IoTDI architecture and framework evaluation using an experimental model of a building, and a comparative evaluation are presented in Section V. In addition, this section presents the results and discussion. Finally, Section VI contains the conclusion and future work.

II. RELATED WORK

The digital twin is an emerging concept used in various fields, including manufacturing, healthcare, and communication [30]. It is one of the key enablers of Industry 4.0 [7], [8], [9], Construction 4.0 [10], and Building 4.0 [11]. As a result, successfully applying the digital twin concept requires overcoming several research challenges. In the context of smart-building digital twins, the integration and management of BIM and IoT data has become a powerful paradigm [20], [31], and the primary focus of research into building digital twins. The remainder of this section focuses on the BIM-IoT integration paradigm and the BIM and IoT integration methods reported in the context of building digital twins.

A. BIM AND IoT DATA INTEGRATION PARADIGM

BIM is a widely adopted technology for maintaining an accurate virtual model of a building to improve the planning, design, construction, and maintenance of a building throughout its life cycle [23], [32], [33]. BIM can also be viewed as a collaboration platform for owners, architects, engineers, contractors, subcontractors, and suppliers by incorporating all aspects and disciplines into a single virtual model [23]. In addition to geometric information, BIM includes functional, parametric, and semantic information about building elements [32]. Due to the prevalence of BIM, numerous proprietary authoring tools are available. In order to facilitate seamless data exchange between BIM authoring tools, an international standardization organization called buildingSMART has introduced the Industry Foundation Classes (IFC) [34] standard data model [35]. The IFC data model is open, public, and non-proprietary, allowing a uniform description of building data [25], [35].

IoT is a novel paradigm shift in information technology. According to Madakam et al. [36], IoT refers to ''an open and comprehensive network of intelligent objects that can auto-organize, share information, data, and resources, reacting and acting in the face of situations and changes in the environment.'' IoT technology is being used in numerous application domains such as smart building, smart cities, smart industries, transportation, and the medical sector [37], [38], [39].

The BIM-IoT paradigm has been an active research area. According to Tang et al. [22], five different methods have been reported for integrating BIM and IoT data. The most widely used method for integrating BIM and IoT data is based on the use of existing APIs (e.g., Revit DB Link, Dynamo, and Grasshopper) provided by BIM tools and relational databases [40], [41], [42]. This method stores BIM and time-series data in a relational database. Integration is achieved using an intermediate schema containing identifier mappings between the physical sensors and the virtual BIM objects. This integration approach is relatively straightforward since it uses an existing API, and the BIM and sensor data are in a relational database. However, repetitive manual exporting is required when the BIM data are modified.

The second type of BIM-IoT integration approach transforms BIM data into a relational database using a new data schema [43], [44], [45], [46]. The new data schema is a simplified version of BIM data based on the user's perspective. Similar to the previous approach, integration is achieved by linking the unique identifiers. This method is suitable for complex BIM models. However, creating a new simplified schema requires significant effort and domain expertise due to the complexity of the IFC data model.

A fully semantic web-based method is also recommended for BIM and IoT data integration [47], [48]. This approach utilizes semantic web technology. The semantic web is an extension of the existing world wide web that uses standards, enabling machine-readable and explicitly defined data on the web [49]. Resource Description Framework (RDF) data format, used to describe information, is a fundamental component of the semantic web. The RDF data format encodes information as triples, which consist of a subject, a predicate, and an object. Each triple component has a distinct Uniform Resource Identifier (URI) [49]. Additionally, the Ontology Web Language (OWL), SPARQL Protocol and RDF Query Language (SPARQL) are employed to represent and query RDF data, respectively [49]. Using Linked Data (LD) principles to publish data is a central concept in the semantic web. These principles promote using RDF, SPARQL, URI, and Hyper Text Transfer Protocol (HTTP) as guiding principles for data publication and querying. The semantic web and linked data principles provide an ideal solution for integrating heterogeneous data using domain ontologies as formal representations of domain knowledge [50]. The key initial step in a fully semantic web-based approach is converting BIM and IoT data into RDF format. The linking is achieved with GUID, and SPARQL is used for queries. This method can enable a full level of semantic interoperability but has limitations. These limitations include extensive data transformation into the RDF format, duplication of sensor data into the RDF store, and the inefficiency of RDF for storing temporal data relative to time-series databases. In addition, sensor readings must be periodically retrieved from the sensor database whenever required and converted to the RDF format. Smart-building digital twins require continuously updated, real-time data, which this method cannot offer.

A hybrid approach is a variant of the semantic web-based method for BIM and IoT data integration [51], [52]. This method stores sensor data in a relational database and uses

RDF to represent other building-related data. Sensor data can be referenced using the sensor identifier contained in the BIM model. This method is advantageous in terms of reducing storage space, data conversion speed, and overall performance. Nevertheless, this method is comparable to partially semantic methods in which sensor observations and associated metadata are not semantically defined, which limits BIM and IoT data interoperability.

B. SMART-BUILDING DIGITAL TWIN ARCHITECTURE

Due to the heterogeneous data environment and multidisciplinary nature of smart buildings, a well-designed digital twin architecture serves as a guiding principle for future research. Accordingly, some researchers have presented architectures [17], [19], [27] and frameworks [14], [28], [29] to address the data integration and management challenges of smart-building digital twins. The primary focus of the proposed architectures and frameworks is the collection, processing, and integration of static and dynamic building data. For instance, Lu et al. [19] presented a digital twin architecture for buildings that included data acquisition, transmission, digital modelling, data/model integration, and service layers. Similarly, other studies [27], [28], [29] used all or a subset of the layers mentioned above with different names but served the same functionalities. Despite their contributions to shaping future research, existing data integration approaches have limitations in enabling semantic interoperability, defined as ''the ability of information systems to exchange information based on shared, pre-established, and negotiated meanings of terms and expressions'' [26]. In addition to the lack of support for semantic interoperability, none of the proposed frameworks [14], [28], [29] have been validated on actual building data.

C. BIM AND IoT DATA INTEGRATION-SMART-BUILDING DIGITAL TWIN CONTEXT

In smart-building digital twin research, BIM and IoT data integration methods can be broadly classified into vendor-specific API-based [27], [53], [54], [55], IFC object mapping (linking)-based [19], [21], [56], and partially semantic-based [14], [17], [57].

The vendor-specific API-based data integration method [27], [53], [54], [55] relies primarily on proprietary APIs provided by BIM authoring tools, such as Revit. In this method, BIM data serves as a virtual model, and sensor readings are linked to it through the API provided by the tool. Despite its ability to combine BIM and IoT data, this method fails to ensure semantic data interoperability since either the IoT data or both BIM and IoT data use implicit data representation without a shared understanding of the meaning of the data.

IFC object mapping (linking)-based methods [19], [21], [56] represent BIM data using the IFC format, while IoT (sensor) data are stored in a relational or document-based database. BIM and IoT data integration is achieved through object mapping tables that relate the Global Unique Identifier (GUID) of sensors defined in the BIM model to the local identifier used by sensor data stores. Whenever IoT data are required, the GUID of the sensor will be retrieved, and the corresponding local identifier will be obtained from the mapping table. The sensor's data can then be read using the sensor's local identifier. Similar to the vendor-specific API-based data integration method, this method fails to facilitate semantic interoperability. Even though the BIM and sensor instances are semantically described using IFC, the sensor observations are still implicitly represented, thereby impeding the full level of semantic interoperability.

Finally, partially semantic integration methods [14], [17], [57] leverage semantic web technologies to integrate BIM and IoT data. The central characteristic of this method is the static data representation using a domain ontology. For instance, Chevallier et al. [17], and Lasitha et al. [57] used domain ontology and RDF data format to represent static building data (BIM) and static descriptions of sensor instances. Sensor readings were retained in time-series databases such as document-based databases. The primary motivations for keeping sensor data in time-series databases are twofold. The RDF data model lacks native support for temporal data since RDF graphs are static snapshots of information [58], [59], [60]. The second argument is that well-structured, mature, and optimized databases for storing and managing time-series data are more efficient than general-purpose graph databases [51], [61]. Similar to the other BIM-IoT integration methods, the partially semantic integration method offers a partial level of semantic interoperability.

D. RESEARCH GAP AND CONTRIBUTION

In general, existing architectures and data integration methods have limitations in supporting the envisioned smart-building digital twin functionalities. The main limitation of existing studies is the lack of support for semantic data interoperability and their inability to keep IoT data in optimal data storage environments. Semantic interoperability ensures the representation of data using explicitly defined domain concepts resulting in machine-understandable data [25], [62]. Smart-building digital twins require semantic interoperability to offer semantic-driven services, such as reasoning, and learning, for decision-making [14], [25]. Furthermore, smart-building digital twins necessitate semantic interoperability to provide a comprehensive solution for enhancing a building's efficiency and functions by leveraging all information related to a building, irrespective of the data format used by building data sources. In addition to lacking support for semantic interoperability, to the best of the authors' knowledge, all existing smart-building digital twin frameworks are conceptual, requiring an experimental evaluation to characterize their feasibility.

In light of the aforementioned research gaps, a smart-building digital twin architecture and framework (BIM-IoTDI) is presented to enable semantic interoperability among smart-building digital twin applications. As a crucial

component of the BIM-IoTDI framework, a BIM and IoT data integration method based on service-oriented architecture, query mediation, and domain ontologies is introduced. The BIM-IoTDI method provides integrated data access for smart-building digital twin applications while retaining IoT (sensor) data in an optimized time-series database. The data integration approach can deliver real-time semantic sensor data utilizing in-memory knowledge graphs to represent raw sensor readings as required. Furthermore, an experimental evaluation model is developed to validate the feasibility of the BIM-IoTDI architecture and framework. The experimental evaluation model is then employed to compare the query response time performance of the BIM-IoTDI method to that of the fully semantic BIM-IoT integration approach.

III. BIM-IoTDI ARCHITECTURE

This study presents a multi-layered architecture called BIM-IoTDI for smart-building digital twins that focuses on integrating static and dynamic building data. The BIM-IoTDI architecture enables semantic interoperability, data storage, and service-oriented data access via an ontology-based data integration method for BIM and IoT data. As shown in Figure 1, the BIM-IoTDI architecture comprises physical, data storage, access, integration, and application layers. This section describes the core layers and functional components within each layer.

A. PHYSICAL LAYER

The physical layer of the BIM-IoTDI architecture represents the physical building. This layer comprises physical subsystems, including sensors and actuators interacting with the smart building environment. Due to the emphasis of this study on the data integration aspect of smart-building digital twins, the physical layer consists of only sensors. Sensors collect useful information by monitoring variables of interest in the building environment, such as temperature, humidity, occupancy, and air quality. The upper layers of the architecture rely on the physical layer's data to provide other smart-building digital twin functions. The physical layer transfers sensor data to a dedicated database in the data storage layer via protocols such as HTTP.

B. DATA STORAGE LAYER

In the BIM-IoTDI architecture, the data storage layer stores static building data, primarily BIM, and dynamic data collected by sensors. The subsequent subsections describe each data source.

1) STATIC BUILDING DATA

Static building data in the storage layer capture less frequently changing building information. This type of building data is typically generated as a BIM model of a building during the design and construction phases. BIM data contain detailed geometric, semantic, and spatial information about building elements, making it an indispensable static data source for smart-building digital twins. Digital twins of smart buildings must integrate static data with dynamic operational data from IoT data sources. Consequently, in the BIM-IoTDI architecture, BIM data and static sensor descriptions are represented as a knowledge graph utilizing domain ontologies and are stored in the smart building knowledge base. Using domain ontologies to represent static building data facilitates semantic interoperability between smart-building digital twin applications. In addition, advanced digital twin features of smart buildings, such as reasoning and decision support, benefit from the knowledge embedded in the knowledge graph. Consequently, the smart building knowledge base serves as a source for all the static data needs of the BIM-IoTDI architecture's upper layers.

2) DYNAMIC BUILDING DATA

In the BIM-IoTDI architecture, the dynamic building data represent the raw sensor readings (IoT Data) collected from the physical building. Due to the cost of managing sensor data as a knowledge graph, the sensor readings are not stored as a knowledge graph, unlike static building data. Since most existing sensor data stores rely on relational or document-based databases, storing sensor data as a knowledge base necessitates duplicating historical and real-time sensor readings. Due to the inherent limitations of graph databases in storing and managing temporal data, it is more efficient to store sensor data in mature, high-performance databases. The BIM-IoTDI architecture retains sensor data in an optimized time-series database for the aforementioned reasons.

C. DATA ACCESS LAYER

The data access layer of the BIM-IoTDI architecture provides the upper layers with access to the static (smart building knowledge base) and dynamic (sensor data) data sources as services. This layer employs a service-oriented approach to facilitate data access via defined interfaces. The knowledge base and IoT (sensor) data as a service are web services that send queries and retrieve results from their respective data stores (i.e., smart building knowledge base and IoT data) in the storage layer. These service interfaces form the foundation for integrated or individual data access within the BIM-IoTDI digital twin architecture for smart buildings.

D. DATA INTEGRATION LAYER

The data integration layer of the BIM-IoTDI architecture provides access to the static and dynamic building data sources. This layer's primary function is to mediate ontology-based queries to static (smart building knowledge base) and dynamic (IoT data) data sources by concealing their heterogeneity. Consequently, the functional blocks within the data integration layer are loosely coupled components of the query mediator. The following paragraphs describe the function and capabilities of each component of the data integration layer's query mediator.

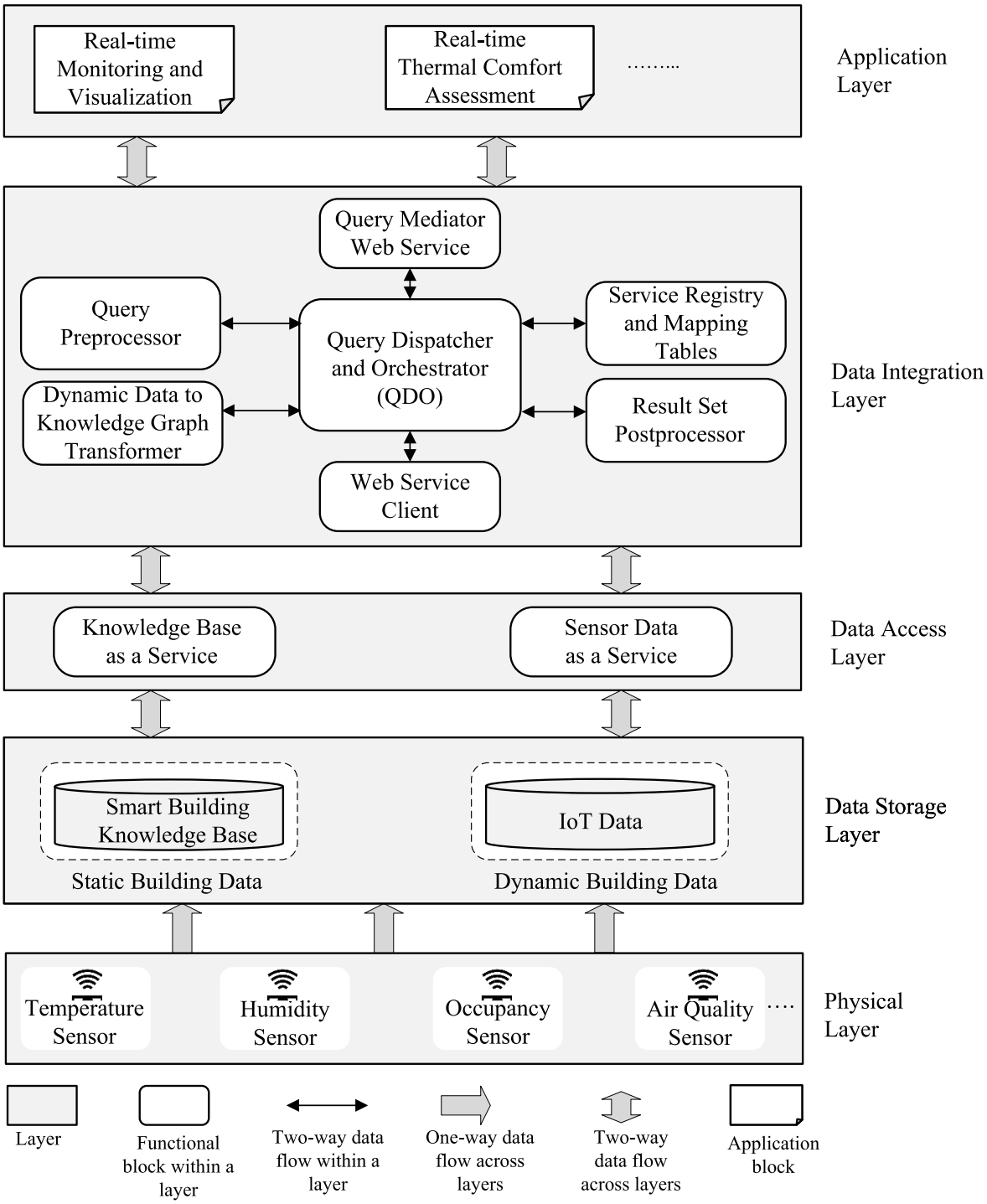


FIGURE 1. BIM-IoTDI architecture for smart-building digital twin.

1) QUERY DISPATCHER AND ORCHESTRATOR (QDO)

The query dispatcher and orchestrator is the query mediator's central component. The primary function of a query dispatcher and orchestrator is to route ontology-based queries to the appropriate data sources and orchestrates the entire query mediation process. Depending on the data requirements of a particular smart-building digital twin application, a query sent to the mediator may need to access either static or dynamic data sources or both. The QDO accepts ontology-based queries via the query mediator web service, plans the query execution, evaluates the query against the data sources, and returns the resulting data.

For query execution planning, QDO uses the query preprocessor to split the input query into static and dynamic

sub-queries based on the expected query syntax and to analyze the interdependency between the sub-queries. The dependency between static and dynamic sub-queries is determined by checking the existence of variables common to both sub-queries. Based on the query dependencies identified by the preprocessor, the orchestrator generates an execution plan for the query. Based on the execution plan, QDO transmits static and dynamic sub-queries to the corresponding data services in the storage layer via the web service client. To execute the dynamic subquery, QDO uses the dynamic data to knowledge graph transformer component, the service registry and mapping tables to convert raw sensor values into a virtual knowledge graph. Additionally, QDO utilizes the result set postprocessor to transform the query result into the correct format. Section V explains the query mediation process in detail.

2) QUERY MEDIATOR WEB SERVICE

The query mediator web service is an endpoint that exposes the query mediator's capabilities to higher-level data consumers. Its primary responsibilities include receiving ontology-based queries, forwarding the queries to the QDO, and returning the result set to the requesting application.

3) QUERY PREPROCESSOR

The components of the query preprocessor provide intermediate query processing tasks to facilitate the execution of queries against the data sources. The query preprocessor has three primary responsibilities. First, it identifies the expected query syntax by comparing it to a predefined template and divides the query into static and dynamic data subqueries per the QDO's request. Second, it determines whether the static and dynamic building data queries are dependent, as described in the explanation of how the QDO operates. Finally, the query preprocessor extracts variable names from the input query for postprocessing or other intermediate operations during query mediation.

4) SERVICE REGISTRY AND MAPPING TABLES

Since sensor data in the BIM-IoTDI architecture are stored in a time-series database, they are inaccessible via ontology-based queries. Therefore, the primary function of the service registry and mapping tables component is to provide the necessary information for retrieving the raw sensor data and transforming it into a knowledge graph. For this purpose, two types of mapping tables are maintained. The first type of mapping table establishes a one-to-one mapping between the virtual sensor identifier (GUID) used by the BIM model and the sensor identifier used by the sensor data web service. The other mapping information is between the sensor database's local schema and the domain ontology's corresponding resource identifier (URI). If multiple dynamic data sources exist, the service registration and mapping table stores the aforementioned information for each data source. Upon request, the service registry and mapping tables component provides mapping information to the QDO.

5) DYNAMIC DATA TO KNOWLEDGE GRAPH TRANSFORMER

The primary function of the dynamic data to knowledge graph transformer is to generate an in-memory knowledge graph from raw sensor readings and mapping information from the service registry and mapping table component. The QDO initiates the construction of an in-memory knowledge graph by sending the raw sensor readings and mapping data. After the in-memory knowledge graph has been constructed, an ontology-based dynamic query can be executed against it.

6) RESULT SET POSTPROCESSOR

The postprocessor component of the BIM-IoTDI architecture handles the final stage of query mediation, which is the combination and formatting of the result set. The postprocessor uses the result sets from the smart building knowledge base and in-memory knowledge graph, as well as a list of projected variables on the input query, to generate a combined result set.

E. APPLICATION LAYER

The application layer of the BIM-IoTDI architecture provides digital twin-based applications. These applications offer services by utilizing integrated data access through the query mediator. This layer includes a variety of applications, such as real-time building monitoring and visualization and real-time thermal comfort assessment.

IV. BIM-IoTDI FRAMEWORK

This section describes the BIM-IoTDI framework, which is based on the BIM-IoTDI architecture and includes a variety of methods for fulfilling the primary functions of each layer of the architecture. The remainder of this section elaborates on the BIM-IoTDI framework.

A. BIM-IoTDI FRAMEWORK-PHYSICAL LAYER

As described in the BIM-IoTDI architecture, the physical layer collects raw sensor readings using various types of sensors. Each sensor transmits its readings to a local gateway that serves as a connector for sensor data storage. This configuration can be accomplished using a Wireless Sensor Network (WSN), in which gateways use Hyper Text Transfer Protocol (HTTP) to send data to cloud storage. Since this study focuses on data integration, it is assumed that a mechanism for transmitting data to dedicated storage is already in place.

B. BIM-IoTDI FRAMEWORK-DATA STORAGE LAYER

In order to create the BIM-IoTDI framework's data storage layer, static and dynamic building data must be maintained in their respective data stores using an appropriate data format. The following subsections describe the methods for constructing the smart building knowledge base (static

TABLE 1. Main namespaces and prefixes used to construct the smart building knowledge base.

Prefix	Namespace
ifc	http://standards.buildingsmart.org/IFC/DEV/IFC4/ADD1/OWL#
bot	https://w3id.org/bot#
sosa	http://www.w3.org/ns/sosa/
brick	https://brickschema.org/schema/Brick#
props	http://lbd.arch.rwth-aachen.de/props#
rdf	http://www.w3.org/1999/02/22-rdf-syntax-ns#
owl	http://www.w3.org/2002/07/owl#
unit	http://qudt.org/vocab/unit/

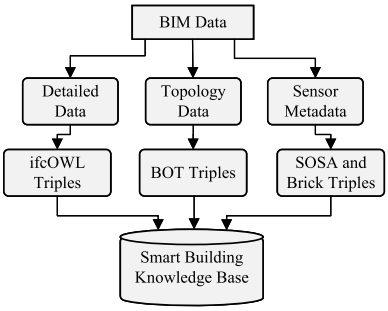


FIGURE 2. Process for constructing smart building knowledge base.

building data) and storing the IoT data (dynamic building data).

1) SMART BUILDING KNOWLEDGE BASE

The smart building knowledge base is the data backbone of the BIM-IoTDI framework. The smart building static data should be represented as a knowledge graph based on domain ontologies and stored as RDF triples in an appropriate data store. Table 1 displays the main namespaces and prefixes of the various ontologies used by the BIM-IoTDI framework to construct the smart building knowledge base.

The smart building knowledge base contains three types of BIM-derived information. The first type of data is detailed data, which includes information about the building's electrical, architectural, and mechanical systems. Topology data, on the other hand, describes building elements and their spatial contexts, such as building stories, spaces, and zones. The third type of information in the smart building knowledge base is sensor metadata, which describes the static description of the sensors. The process of creating a smart building knowledge base in the BIM-IoTDI framework is depicted in Figure 2.

The as-built BIM model of a building is the primary source of static data. As-built BIM data can be converted into a static graph model using standalone data converter tools or plugins in BIM authoring tools. As depicted in Figure 2, the smart building knowledge base must be created by converting BIM data into RDF triples using appropriate domain ontologies. The BIM data must first be converted to standard IFC schema-based data to achieve this objective. Using the IFC data as input and ifcOWL [63] as domain ontology, the detailed building data (i.e., electrical, architectural, and mechanical) information must be converted to ifcOWL RDF triples and property sets from the IFC must be described using the PROPS [1] ontology.

Even though BIM data contain information regarding the building's topology, it is difficult to navigate owing to the complexity of the IFC schema. Therefore, in the BIM-IoTDI framework, a simplified and more user-friendly ontology called Building Topology Ontology (BOT) [64] is used to describe topological information.

In addition to detailed building data and topology information, the sensor's static metadata should be represented by two ontologies, namely SOSA [65], and Brick [66]. Static metadata about sensors refers to any information about the sensor that does not change over time. Using terms from the SOSA ontology, for instance, the sensor can be labelled as a sensor instance, and the variable it observes can be stated explicitly. In addition, the types of sensors and the units can be identified using terms from the Brick [66] and QUDT ontology [67], respectively. This static sensor information can be extracted from the BIM model if its level of detail permits it, or it can be obtained from the Building Management System (BMS). RDF triples representing detailed data, topology data, and sensor metadata must be combined into a single knowledge graph and stored in RDF storage.

Figure 3 illustrates a high-level view of the smart building knowledge base's data structure displaying only a subset of the topology and sensor meta-data-related ontology terms for a specific sensor within a room. As shown in Figure 3, the topology information is represented hierarchically, where a given building story, for example *inst:level* where *inst* represents the base URI, is associated with *bot:Storey* and *ifc:BuildingStorey* terms via a *rdf:type* predicate. In addition, the building story instance has a Global Unique Identifier (GUID), and name literals defined by *props:IfcGuid* and *props:Name* predicates, respectively. The spaces within the story are also associated with a space instance via *bot:hasSpace* property. A given room on a particular building story is defined as a type of *ifc:Space*, and *bot:Space*, to relate it to the topology terms and *sosa:FeatureOfInterst* to indicate that sensors observe some properties inside the room. Each sensor is also represented by *sosa:Sensor*, *bot:Element*, and *ifc:Sensor* in order to identify it as a sensor instance. The property observed by the specified sensor is indicated by *sosa:Observes* predicate, and the property instance is defined as a type of *sosa:ObservableProperty*. In addition, the sensor type is labelled as *brick:Temperature_Sensor*. Finally, the unit of the property observed by the sensor is defined using *brick:hasUnit* as the predicate and unit ontology terms such as *unit:DEG_C* to indicate that the unit of measurement for the given temperature sensor instance is degree centigrade.

Figure 3 shows the power of domain ontologies to connect cross-domain data from BIM and IoT sources. A given sensor's spatial context and other static characteristics are explicitly defined. Dynamic data associated with static

[1] https://github.com/w3c-lbd-cg/props

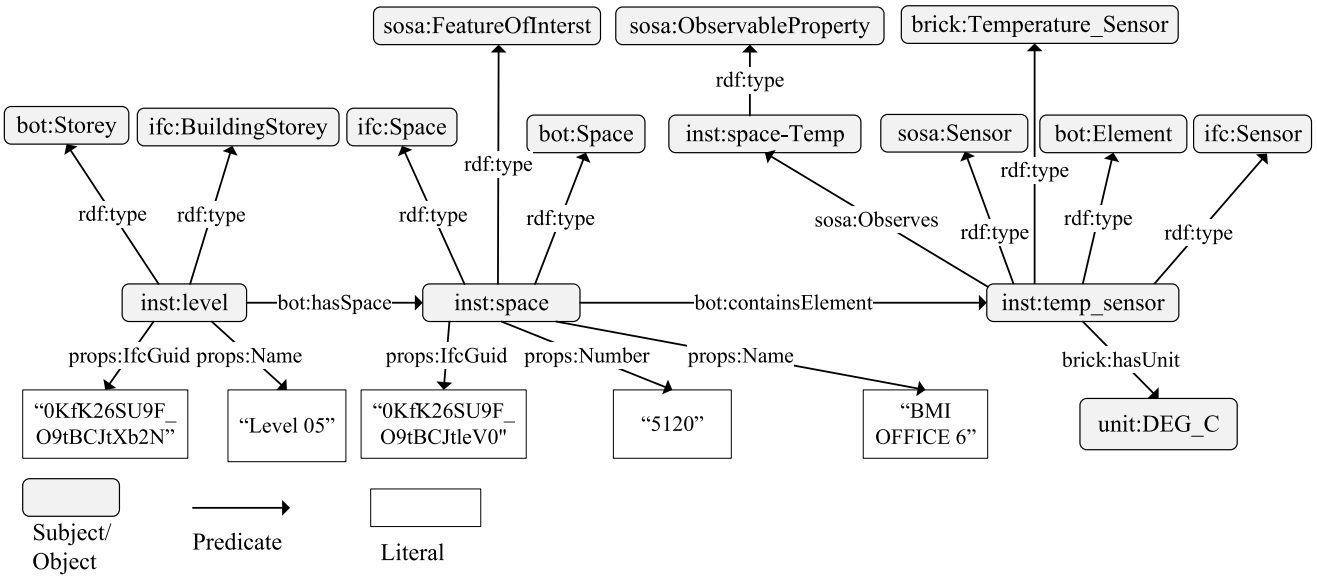


FIGURE 3. High-level view of the data structure for a sensor-equipped room in a specific building.

sensor information become more expressive than raw sensor observation by providing a spatiotemporal context.

2) IoT (SENSOR) DATA

The dynamic building data are saved in the BIM-IoTDI framework's IoT (Sensor) database. Since sensor data evolve over time, both historical and real-time observations must be stored in efficient time-series data stores, such as document-based databases.

C. BIM-IoTDI FRAMEWORK-DATA ACCESS LAYER

The data access layer of the BIM-IoTDI framework includes a knowledge base and sensor data web services that provide static and dynamic data access. The descriptions of both data services are provided below.

1) KNOWLEDGE BASE AS SERVICE

The knowledge base as a service is a web service that provides access to static building data. This web service should be a SPARQL endpoint capable of executing SPARQL queries against a knowledge base and returning JavaScript Object Notation (JSON) objects containing the results. The web service for the knowledge base must support Create, Read, Update, and Delete (CRUD) operations that can be achieved by sending an appropriate SPARQL query as an HTTP request.

2) SENSOR DATA AS A SERVICE

Sensor data as a service in the BIM-IoTDI framework is a web service for accessing dynamic building data. This web service provides access to historical as well as real-time readings of a sensor identified by its unique identifier. For this purpose, Web services based on the Representational State Transfer (REST) API are ideal. The web service should offer read-only REST endpoints for retrieving sensor data given a date and time range as query string parameters and should return JSON formatted data.

D. BIM-IoTDI FRAMEWORK-DATA INTEGRATION LAYER

The query mediator for static and dynamic building data sources, which is part of the data integration layer, is one of the significant contributions of this work. The objective is to provide integrated data access for smart-building digital twin services through ontology-based queries while maintaining the dynamic data source (sensor data) in its optimal time-series database. Before describing the query mediation process, the query syntax expected by the query mediator should be defined.

Since BIM-IoTDI employs semantic web technologies, the query mechanism is based on the SPARQL Protocol and RDF Query Language (SPARQL). SPARQL is only compatible with RDF triple stores accessible through SPARQL endpoints. Therefore, it can be used to read or write information to a smart building knowledge base. However, SPARQL cannot be used to access sensor data as it is still stored in a time-series database accessed via a REST API endpoint which doesn't support SPARQL query execution.

A SPARQL query operation to access the sensor data web service should be available to retrieve sensor data using ontology-based queries. Therefore, a new query extension is introduced into an existing SPARQL operation known as *SERVICE* [68], which is used to execute SPARQL queries on remote SPARQL endpoints via the HTTP protocol. The main objective of extending the *SERVICE* operation is to permit the execution of SPARQL queries on raw sensor data as if the conventional web service is a SPARQL endpoint. Extending existing SPARQL operations with minor modifications facilitates the adaptability of the extended query syntax.

```
SERVICE <WSU?{?SIV}...> {
    SELECT ?v1 ?v2 ?v3 ... ?vn
        WHERE {
            DDGP1.
            DDGP2.
            DDGP3.
            ...
            DDGPn.
         }
         QM
}
```

LISTING 1. Syntax of the new extended SPARQL *SERVICE* query.

This approach was also used by Mosser et al. [69]. However, unlike the method proposed by Mosser et al. [69], this study's approach is based on a different query syntax, does not require navigating the JSON response inside the extended SPARQL query, and uses in-memory virtual knowledge graphs to support the execution of any SPARQL query against the sensor data.

The new extended query syntax is shown in Listing 1. The new extended *SERVICE* query syntax requires the Web Service URI of the sensor web service, which is represented as *WSU*. The *WSU* stores the base REST service URI for reading sensor data. Next to *WSU* on the extended query syntax is the query string parameter represented as (*?{?SIV}...*) on the extended query syntax. Within the curly braces of the query string is a required SPARQL Identifier Variable (*?SIV*). The term *?SIV* is the name of the SPARQL variable containing the GUID of the sensor, which can be retrieved from the static building data. This value of *?SIV* may also be entered directly between the curly braces. The remaining portion of the query following *?{?SIV}* may include date time ranges or specific time stamps as query parameters for HTTP requests to the sensor web service.

The SPARQL query to be executed on the sensor data following its conversion into an in-memory knowledge graph is contained within the body of the extended *SERVICE* query syntax. Thus, the body of the extended *SERVICE* query contains a SPARQL *SELECT* statement, with a list of projected variables $?v_1$, $?v_2$, $?v_3$, ..., $?v_n$, and a *WHERE* clause containing a Dynamic Data Graph Pattern ($DDGP_1$, $DDGP_2$, $DDGP_3$ ..., $DDGP_n$). DDGP specifies the set of graph patterns that must be matched against the sensor data knowledge graph. Additionally, the extended query syntax can also be used with optional query modifiers (*QM*). Listing 2 provides an example of extended *SERVICE* query syntax. The given example query retrieves the sensor reading values and timestamps within a specified time interval for a particular sensor whose GUID is stored in the *?guid* SPARQL variable.

A new extension of *SERVICE* query syntax is designed to retrieve dynamic building data (sensor data). The static building data are accessible via a standard SPARQL query. Therefore, a SPARQL query from a requesting application may contain either Static Data Query (SDQ), Dynamic Data Query (DDQ), or both. In the case of a combined query, Listing 2 shows the integrated data access query syntax expected by the query mediator in the BIM-IoTDI framework.

```
SERVICE<http:localhost:5000/reading?{?guid}&
    start=2021-01-01T01:05:00&end=2021-02-01
    T01:05:00> {
   SELECT ?time ?value
       WHERE {
           ?obs rdf:type sosa:Observation.
           ?obs sosa:hasSimpleResult ?value.
           ?obs sosa:resultTime ?time.
        }
  }
```

LISTING 2. Example query using the extended *SERVICE* query syntax.

```
PREFIX prefix: <URI>
...
BASE <Base URI>

SELECT ?vp1 ?vp2 ?vp3 ... ?vpn
    WHERE {
        SDGP1.
        SDGP2.
        SDGP3.
        ...
        SDGPn.
        SERVICE <WSU?{?SIV}...> {
            SELECT ?v1 ?v2 ?v3 ... ?vn
                WHERE {
                    DDGP1.
                    DDGP2.
                    DDGP3.
                    ...
                    DDGPn.
                 }
                 QM
        }
    }
```

LISTING 3. SPARQL query syntax for integrated data access.

The first part of the integrated data access query, as shown in Listing 2, is the mandatory definition of the base URI used in the smart building knowledge base and the definition of the required namespaces and prefixes. Following the namespaces and prefixes is the top-level *SELECT* statement, which declares the requested projected variables (i.e., $?vp_1$, $?vp_2$, $?vp_3$, ..., $?vp_n$). To retrieve static data, the list of Static Data Graph Pattern ($SDGP_1$, $SDGP_2$, $SDGP_3$ ..., $SDGP_n$) statements must be matched against the smart building knowledge base within the top-level *WHERE* clause. The static data graph patterns are expected to specify the spatial context of the sensor from which values are to be read. After defining the spatial context, the static graph pattern must retrieve the sensor's GUID and store it in a SPARQL variable. The SPARQL variable will then be substituted in place of *?SIV* in the dynamic data subquery. Following the static data graph pattern is the previously described extended *SERVICE* query syntax.

1) THE QUERY MEDIATION ALGORITHM

On the basis of the integrated data access query syntax, it is now possible to describe the query mediation process of the BIM-IoTDI framework. The process of answering a given SPARQL query by the mediator is outlined in Algorithm-1. The query mediation process begins when a SPARQL Query (SQ) is received via the query mediator web service. The Query Dispatcher and Orchestrator (QDO) then forwards the query to the preprocessor for analysis. The query preprocessor examines the received query, checks whether it contains the extended *SERVICE* query syntax (***Algorithm-1:line 1***), and returns the result to the QDO.

Whenever the input query (SQ) contains the extended query syntax (***Algorithm-1:line 2***), the QDO splits it into static data query (SDQ) and dynamic data query (DDQ) components using the query preprocessor (***Algorithm-1:line 3***). The next step is to analyze the dependency between SDQ and DDQ in order to generate a query execution plan (***Algorithm-1:line 4***). The SDQ-DDQ dependency is determined by examining whether the DDQ uses the SPARQL variables declared by the SDQ. When the DDQ uses a variable from the SDQ, it implies that the DDQ requires the variable's value after the SDQ has been executed, indicating their interdependence. If SDQ and DDQ are dependent (***Algorithm-1:line 5***), the DDQ must have a sensor identifier in order to retrieve data from the sensor web service. Consequently, the QDO dispatches and executes the SDQ on the smart building knowledge base via the web service client (***Algorithm-1:line 6***). When executed, the SDQ returns the GUID of the sensor used by the smart building knowledge base. The identifier used in the smart building knowledge base should then be mapped to the corresponding sensor identifier utilized by the sensor data web service. For this purpose, QDO refers to the service registry and mapping tables to read the corresponding sensor identifier used by the sensor data web service (***Algorithm-1:line 7***). The next step involves substituting the sensor identifier value into the DDQ service URI (***Algorithm-1:line 8***) and retrieving the raw sensor data using the web service client (***Algorithm-1:line 9***).

Before executing the query body of the DDQ, the retrieved raw sensor data, described according to the local schema of the sensor data service, must be converted into a knowledge graph in real time. Converting raw sensor data into a knowledge graph is essential for enabling semantic interoperability. In order to construct a knowledge graph of the sensor data in real-time, mapping from the local schema properties to the global sensor data ontology concepts is required. Therefore, the QDO refers to the local schema to global schema ontology mapping tables from the service registry and mapping tables component and retrieves the URI of the corresponding ontology concepts for each property in the local schema of the sensor data (***Algorithm-1:line 10***). The QDO then sends the mapping rules, the raw sensor data, and the base URI to the dynamic data to the knowledge graph transformer component of the query mediator. Using the mapping rules raw sensor data and base URI, the dynamic data to knowledge graph transformer builds an in-memory virtual knowledge graph in real-time (***Algorithm-1:line 11***).

Algorithm-2 shows the process of transforming dynamic data into an in-memory virtual knowledge graph. The process begins by creating an empty virtual knowledge graph (***Algorithm-2:line 1***), then iterates through the sensor observation objects (***Algorithm-2:line 2***), creating RDF triples for each sensor observation instance and attribute (***Algorithm-2:line 3-10***), inserting the triples into the virtual knowledge graph (***Algorithm-2:line 5 and line 9***), and returning the result (***Algorithm-2:line 12***).

In the preceding steps, the real-time conversion of raw sensor data to a virtual knowledge graph enables the execution of ontology-based SPARQL queries on the sensor data. Therefore, QDO executes the dynamic data query (DDQ) using a SPARQL query engine that manages the in-memory knowledge graph and retrieves the result (***Algorithm-1:line 12***).

In the case of extended query input, the last step of the query mediation process involves combining the result sets from the static data query (SDQ) and dynamic data query (DDQ). The content of the output result set is determined by the projected variables of the *SELECT* statement at the top level of the original input query (SQ). Therefore, the QDO extracts the projected variables using the query preprocessor (***Algorithm-1:line 13***) and then sends the projected variables, along with the result sets of SDQ and DDQ, to the postprocessor. The postprocessor component returns to the QDO after combining the result sets (***Algorithm-1:line 14***). The QDO then sends the query solution (i.e., *RRS*) to the requesting application through the mediator web service (***Algorithm-1:line 15***).

The original query to the mediator may include the extended *SERVICE* query syntax while SDQ and DDQ are independent (***Algorithm-1:line 17***). In this case, the DDQ is not required to wait until SDQ has been executed. Therefore, SDQ (***Algorithm-1:line 18***), and DDQ (***Algorithm-1:line 19-22***) will be executed simultaneously. Based on the projected variables (***Algorithm-1:line 23***), the combined result set (***Algorithm-1:line 24*** (i.e., *RRS*) is returned (***Algorithm-1:line 25***).

The query mediation process described above applies to extended input queries. If the input query does not contain the extended query syntax (***Algorithm-1:line 28***), the mediator executes it on the smart building knowledge base and returns the result set (i.e., the *RRS*) to the requesting application (***Algorithm-1:line 30***).

E. BIM-IoTDI FRAMEWORK-APPLICATION LAYER

The application layer of the BIM-IoTDI framework represents any application that consumes data through the query mediator of the data integration layer. Each application sends a request that contains a SPARQL query. The request should be addressed to the REST service endpoint of the query mediator. Applications are expected to be independent

Algorithm 1 Ontology-Based Query Mediation Algorithm

Input: SPARQL Query (*SQ*)
Output: RDF Result Set (*RRS*)

```
Check if SQ contains extended SERVICE query
if SQ contains extended query then
    Break SQ into (SDQ) and (DDQ)
    Check if SDQ and DDQ are dependent
    if SDQ and DDQ are dependent then
        Execute SDQ
        Retrieve the sensor Id
        Substitute the sensor Id into DDQ URI
        Retrieve the raw sensor data
        Retrieve the URI of sensor ontology concepts
        Build sensor data virtual knowledge graph
        Execute DDQ
        Extract projected variables from SQ
        Combine SDQ and DDQ result sets
        return Combined RRS to the application
    end
    else if SDQ and DDQ are not dependent then
        Execute SDQ
        Retrieve the raw sensor data
        Retrieve the URI of sensor ontology concepts
        Build sensor data virtual knowledge graph
        Execute DDQ
        Extract projected variables from SQ
        Combine SDQ and DDQ result sets
        return Combined RRS to the application
    end
end
else if SQ does not contain extended query then
    Execute SQ
    return RRS to the application
end
```

Algorithm 2 Algorithm for Dynamic Data to in-Memory (Virtual) Knowledge Graph Transformer

Input: Raw Sensor Data (*RSD*), Mapping Rules (*MR*), Base URI (*BURI*)
Output: Virtual Knowledge Graph (*VKG*)

```
Initialize empty VKG
foreach Sensor Observation Object (SOO) in (RSD) do
    Create a URI for sensor observation using the BURI,
     timestamp, and sensor Id.
    Create RDF triple for the sensor observation
     instance
    Insert the triple into VKG
    foreach Key in SOO do
        Read the corresponding URI from MR
        Create RDF triple
        Insert the triple into VKG
    end
end
return VKG
```

services to preserve the service-oriented nature of the BIM-IoTDI architecture.

V. EVALUATION

An experimental evaluation model based on an actual building as a case study is developed to characterize the viability of the BIM-IoTDI architecture and framework. The experimental evaluation model is then used for conducting a comparative evaluation. The comparative evaluation compares the performance of BIM-IoTDI and fully semantic data integration methods. The remainder of this section elaborates on the development of the evaluation model and the comparative evaluation.

A. CASE STUDY

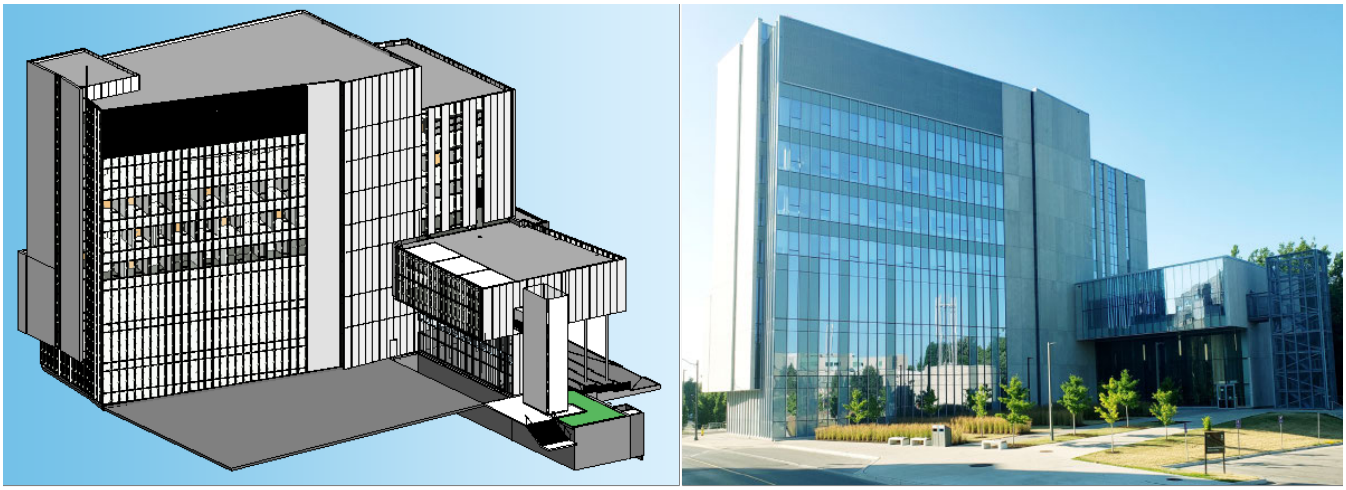
The case study is conducted on a building at Western University's main campus in Canada. The specific building selected, Western Interdisciplinary Research Building (WIRB), is shown in Figure 4-b. The WIRB[2] is a 118,000 sq. ft. seven-story facility containing classrooms, research spaces, public amenities, and public spaces. Figure 4-a depicts a virtual building model extracted from the building's BIM model, which comprises detailed architectural, structural, electrical, mechanical, and plumbing information. Following the procedures specified by the BIM-IoTDI framework, the implementation of each layer of BIM-IoTDI architecture is discussed.

1) WIRB-DATA STORAGE LAYER

Implementing the data storage layer requires the creation of a knowledge base and sensor data store for the smart building. In order to create the smart building knowledge base, topological, detailed, and sensor metadata from the BIM model are extracted and converted as the first step. Figure 5 depicts converting the BIM model to ifcOWL triples containing the detailed building data. Initially, the WIRB-BIM data are exported to IFC version 4 using the Revit BIM authoring tool. The default IFC export settings are modified to correctly export the sensor instances already present in the WIRB-BIM data. Using the WIRB-IFC data as input and ifcOWL [63] as a domain ontology, the detailed building data are converted into ifcOWL RDF triples using theIFCtoRDF tool. The resulting RDF data are serialized into turtle (.ttl) format.[3]

To generate the BOT triples describing topological information, a direct conversion is performed using a Revit plugin called revit-bot-exporter [4] [70] as shown in Figure 6. To export the sensor metadata from the BIM model, a revit-sosa/brick-exporter plugin is developed that analyzes the

[2]https://www.uwo.ca/bmi/about/wirb.html
[3]https://github.com/pipauwel/IFCtoRDF
[4]https://github.com/MadsHolten/revit-bot-exporter



a. Virtual building model

b. Physical building

FIGURE 4. Western Interdisciplinary Research Building (WIRB) used for the case study.

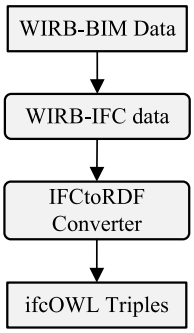


FIGURE 5. Conversion process for detailed building data to ifcOWL triples.

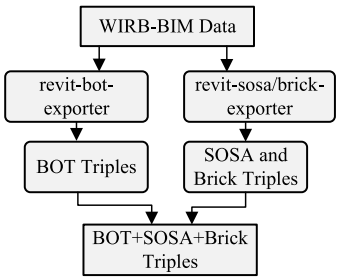


FIGURE 6. Conversion process for topological and sensor metadata to BOT, SOSA, and Brick triples.

intersection between each room's closed shell solid and the sensor instance's solid to determine the sensor's location and generates SOSA and Brick triples.

Finally, the ifcOWL, BOT, SOSA, and Brick triples are combined into a single turtle (.ttl) file and uploaded to Apache Jena's[5] persistent triple store known as TDB. The final resulting smart building knowledge base contained 30,448,809 triples.

[5]https://jena.apache.org/

After creating the smart building knowledge base, the next step involved creating the sensor data source. Due to the unavailability of actual sensor readings, a program is developed to generate sensor readings for various sensor types. The generated sensor data are then stored in the MongoDB NoSQL database as a time-series collection. Each sensor reading includes a unique sensor identifier, a timestamp, and the sensor reading's actual value. The sensor data generator program generated 525,600 minutely historical readings representing one year of data for each sensor.

2) WIRB-DATA ACCESS LAYER

Implementing the data access layer requires the development of sensor and knowledge base web services. The web service for sensor data is implemented using REST API. The web service provides historical sensor data within a specified timestamp interval, at a given instant, or the most recent reading. In addition, it simulates real-time data collection by continuously generating sensor readings minutely using the sensor data generator and storing them in the sensor database. The sensor data are accessible via GET requests with a query string containing the parameter values, and the response is in a JSON format. Finally, the web service is hosted on a local Node.js server.

The knowledge base web service is hosted on the Fuseki server of Apache Jena. This server provides HTTP access to the smart building knowledge base. It accepts the SPARQL query, executes it against the persistent triple store (knowledge base) and returns a JSON response containing the result set. The knowledge base web service is also hosted on a local server in this case study.

3) WIRB-DATA INTEGRATION LAYER

The data integration layer (query mediator) is implemented using Java and the Java API from Apache Jena. Each

functional component of the query mediator is implemented as a Java class. The Spring Boot Java framework is used to implement the query mediator's REST web service and access other web services through a web service client. HTTP POST requests with SPARQL queries in the request body can be used to access the query mediator web service. Like other web services, the query mediator web service is hosted locally.

4) WIRB-APPLICATION LAYER

In the case study building, a real-time monitoring and visualisation application is developed to assess the viability of the BIM-IoTDI framework. This use case illustrates one of the most significant applications of digital twins for smart buildings. For 3D visualization of the case study building, an open-source JavaScript 3D graphics Software Development Kit (SDK) named xeokit[6] is used. xeokit renders 3D files in a web browser using the binary 3D file format called XKT, which enables a rapid rendering of large 3D models. The XKT data format also includes IFC file metadata such as GUID, name, and parent.

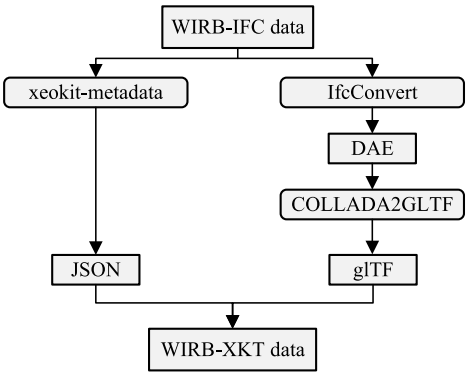


FIGURE 7. Conversion process for IFC to XKT (Adopted from:[7]).

As shown in Figure 7, open-source command-line tools are used to convert WIRB-IFC data into XKT format. Once the XKT file is generated, a web application is created to visualize the case study building's 3D model. In the 3D model, sensors inside a given room are represented by dynamically generated 3D markers. The user can then select a marker to access the real-time or historical sensor data.

The application for real-time monitoring continuously interacts with the query mediator to retrieve the required data. When a user selects the markers displayed on the 3D model, a SPARQL query is sent to the mediator to populate the dashboard with a list of available sensors. Listing 4 is an example of a SPARQL query used by the monitoring application to retrieve all sensors in a particular room. The query specifies the spatial context by selecting the stories and rooms in the building, then filters sensor instances within that space.

[6]https://xeokit.io/index.html
[7]https://www.notion.so/Converting-IFC-Models-to-XKT

```
SELECT  ?sensorType WHERE {
    ?storey rdf:type bot:Storey.
    ?storey props:Name "LEVEL 05".
    ?storey bot:hasSpace ?room.
    ?room props:number "5170".
    ?room bot:containsElement ?sensor.
    ?sensor rdf:type ?sensorType.
    FILTER regex(str(?sensorType), 'brick').
}
```

LISTING 4. SPARQL query example for retrieving sensor types within a room.

```
BASE <https://linkedbuildingdata.net/ifc/
   resources20211224_020724/>
SELECT ?result ?time WHERE {
    ?storey rdf:type bot:Storey.
    ?storey props:Name "LEVEL 05".
    ?storey bot:hasSpace ?room.
    ?room props:number "5170".
    ?room bot:containsElement ?sensor.
    ?sensor rdf:type  ${sensorType}.
    ?sensor props:ifcGuid ?guid.
    SERVICE <http://localhost:5000/api/v1/
   readings?sensor_id={?guid}> {
    SELECT ?result ?time WHERE {
        ?obs rdf:type sosa:Observation.
        ?obs sosa:hasSimpleResult  ?result.
        ?obs sosa:resultTime ?time.
       }
    }
 }
```

LISTING 5. Example SPARQL query for retrieving sensor readings in real-time.

The application retrieves the actual sensor readings and displays them on a dashboard after identifying the sensor entities in each building space. The sensor readings can be real-time or historical. The application sends integrated data access queries to the query mediator on a continuous basis in order to retrieve real-time sensor data. Listing 5 is an example of a SPARQL query that requests the readings of a sensor located at floor level 05, room number 5170, with a sensor type defined by the (*sensorType*) string variable such as a temperature sensor or occupancy sensor, based on the user's selection. As required by the extended query syntax, the static graph pattern retrieves the GUID of the sensor, assigns it to the *?guid* SPARQL variable, and embeds it within the sensor service URI so that it can be used to read the corresponding sensor identifier when executing sensor web service call.

Likewise, the monitoring application retrieves historical sensor readings for a specified time interval. Listing 6 demonstrates an example of a SPARQL query for retrieving historical sensor readings via an integrated data access query. In this query, the sensor identifier is first extracted based on the specified spatial context, followed by the extraction of sensor data from the sensor data service using the start time (*startDateTime*) and end time (*endDateTime*) values as

```
BASE <https://linkedbuildingdata.net/ifc/
    resources20211224_020724/>
SELECT ?result ?time WHERE {
    ?storey rdf:type bot:Storey.
    ?storey props:Name "LEVEL 05".
    ?storey bot:hasSpace ?room.
    ?room props:number "5170".
    ?room bot:containsElement ?sensor.
    ?sensor rdf:type ${sensor Type}.
    ?sensor props:ifcGuid ?guid.
    SERVICE <http://localhost:5000/api/v1/
    readings?sensor_id={?guid}&startDateTime=
    ${startDate}&endDateTime=${endDate}> {
    SELECT ?result ?time WHERE {
        ?obs rdf:type sosa:Observation.
        ?obs sosa:hasSimpleResult  ?result.
        ?obs sosa:resultTime ?time.
      }
    }
}
```

LISTING 6. Example SPARQL query to retrieve historical sensor readings.

TABLE 2. Summary of datasets used for the evaluation.

Dataset	Number of Data Points
Smart Building Knowledge Base	30,448,809 triples
Semantic Sensor Data	1,048,320 triples
Raw Sensor Data	525,600 data objects

temporal filters. The application then parses the RDF result set and shows the resulting data.

B. COMPARATIVE EVALUATION

The BIM-IoTDI method is evaluated by comparing it with a fully semantic data integration method [47], [48]. The following sections elaborate on the specifics of the evaluation methodology. The primary objective of the evaluation is to identify the advantages and disadvantages of the fully semantic approach and BIM-IoTDI in a smart-building digital twin context.

1) DATASETS

Three types of data are used for the evaluation. The first type of data is the knowledge base for smart buildings, which is also used for the case study. This dataset contains a large amount of RDF triples detailing the case study building's architectural, electrical, and mechanical systems. The remaining data consisted of sensor readings with semantic descriptions represented as RDF triples and raw sensor readings represented as JSON objects. Both raw and semantic sensor data contain one year of minutely generated sensor readings. Table 2 provides a summary of all the datasets used for evaluation.

2) EXPERIMENTAL SETUP

This research's experiments are conducted on a personal computer. Table 3 summarizes the machine's specifications in detail.

TABLE 3. Specifications of the machine used in the experiments.

Property Name	Specifications
Processor	Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz
Installed RAM	16.0 GB (15.8 GB usable)
Operating System	Windows 10

```
    SELECT  ?result ?time WHERE {
        ?storey rdf:type bot:Storey.
        ?storey props:Name '''LEVEL 05'''.
        ?storey bot:hasSpace ?room.
        ?room props:number '5170'.
        ?room bot:containsElement ?sensor.
        ?obs rdf:type sosa:Observation.
        ?obs sosa:madeBySensor ?sensor.
        ?obs sosa:hasSimpleResult  ?result.
        ?obs sosa:resultTime ?time.
    FILTER(?time >= "${startDateTime}"^^xsd:
    dateTimeStamp && ?time<="${endDateTime
    }"^^xsd:dateTimeStamp).
    }";
```

LISTING 7. SPARQL query used for the baseline experiment.

3) EVALUATION METRIC

Query Response Time (QRT) is the metric used to evaluate the performance of the fully semantic and the BIM-IoTDI methods. Since smart-building digital twins require real-time updated data, query response time is used as an evaluation metric. The query response time indicates the amount of time required to execute and return the results of a given query. In this study, the QRT is measured from the time a SPARQL query is received to the time a result set is returned to the requesting entity.

4) EXPERIMENTS

Four experiments are conducted during the comparative evaluation. The three experiments involved implementing a fully semantic data integration method on the Blazegraph, GraphDB, and Apache Jena graph databases. The sensor readings are initially represented as RDF triples for the initial three experiments and then combined with the smart building knowledge base. The combined knowledge base provides a centralized access point for both static and dynamic building data. The SPARQL query shown in Listing 7 is used as a test query for evaluating the query response time of the fully semantic implementations. The query shown in Listing 7, first identifies the spatial context of a given sensor, selects the observations made by the sensor, and finally filters the sensor readings between a given start (*startDateTime*) and end date-time (*endDateTime*).

On the other hand, the fourth experiment evaluates the BIM-IoTDI method. In this case, only the static building data is represented as RDF triples, whereas the raw sensor data are stored as a collection of data objects in a NoSQL database. Whenever sensor data are required, they are converted to RDF triples in real-time by the query mediator. The query used for evaluating the performance is similar to that of the

```
inst:2DYigUjePF4x$FhiZp78Lz_2021-02-03
   T222500.000Z a sosa:Observation ;
        sosa:madeBySensor inst:
   sensor_2DYigUjePF4x$FhiZp78Lz;
        sosa:hasSimpleResult "25.45"^^xsd:
   double;
        sosa:resultTime "2021-02-03T22
   :25:00.000Z"^^xsd:dateTimeStamp .
```

LISTING 8. Example of a semantically described sensor observation.

SPARQL query, as shown in Listing 6. Similar to the first three experiments, the first part of the query identifies a specific sensor's spatial context. After identifying the spatial context, it extracts the unique identifier from the smart building knowledge base and uses the extended version of the SPARQL query to retrieve sensor readings as RDF triples.

In each experiment, the QRT is calculated by retrieving a different number of sensor readings. The number of observations varied between the intervals [10,100] by tens, [100,1000] by hundreds, [1000,10000] by thousands, and [10000,525000] by tens of thousands. Additionally, the QRT required to retrieve a single sensor reading is determined and recorded.

C. RESULTS AND DISCUSSION

1) CASE STUDY

Figure 8 shows the outcome of the monitoring application retrieving real-time temperature readings from Room-5170. The BIM-IoTDI method is suitable for real-time digital twin applications, as demonstrated by the application's real-time monitoring functionality. Real-time sensor observations are described with domain ontology while stored in optimal time-series storage, which is crucial for offering a full level of semantic interoperability. Listing 8 illustrates a sensor observation described using domain ontology during the process of query mediation. As shown in Listing 8, unique URIs are generated for semantic sensor observations by combining the base URI, the sensor's GUID, and the time stamp. Similarly, each sensor instance is identified by a URI generated by concatenating the base URI with the sensor's GUID.

2) COMPARATIVE EVALUATION

Figure 9 and Figure 10 depict the comparative evaluation results. In order to compare the results of BIM-IoTDI with those of the fully semantic implementations on Apache Jena and Blazegraph, it is possible to consider three crucial aspects of the graphs. The first portion of the graph is where the number of observations falls between the ranges shown in Figures 9 and 10 ([1,100000) and [100000,200000), respectively. In this case, there is a substantial performance gap between the BIM-IoTDI and the fully semantic approach (baseline). It takes nearly nine and six seconds to read a single sensor observation using the fully semantic approach with Apache Jena and Blazegraph. Using the BIM-IoTDI method, the QRT is approximately 37 milliseconds. Inadequate indexing by the graph databases and the size of the knowledge graph could be contributing factors to this significant performance difference. In this regard, the BIM-IoTDI method extracts only the spatial context from the knowledge base and uses the NoSQL sensor database's quick query execution, thereby significantly reducing the QRT. Despite the overhead incurred by the query mediation process, the BIM-IoTDI integration approach outperformed both implementations, even when reading a large number of sensor readings, up to 200,000 at once. This demonstrates that the BIM-IoTDI method is better suited for smart-building digital twin real-time updated data needs while semantically representing the sensor observations.

As the number of retrieved observations increases in the range [1,100000) depicted in Figure 9 and [100000,200000) depicted in Figure 10, the QRT of Apache Jena and Blazegraph implementations increases at a slower rate than that of BIM-IoTDI. The performance gap reflects the overhead caused by the time required to construct the in-memory sensor data knowledge graph and postprocess the result set.

Figure 10 demonstrates that when the number of retrieved observations is 200000, BIM-IoTDI has comparable QRT values to Blazegraph and Apache Jena fully semantic implementations. As shown in Figure 10, the third section of the graph is where the number of observations falls between 200,000 and 520,000. BIM-IoTDI's query response time in this scenario increases much more rapidly than fully semantic implementations. There are two primary causes for this disparity in performance. First, in the current version of the BIM-IoTDI integration method, the real-time transformation of sensor data to an in-memory knowledge graph is not parallelized, incurring an increased cost as the amount of data retrieved increases. Other factors that gave both Blazegraph and Apache Jena an advantage are the SPARQL query engine's advanced query optimization and the fact that static and dynamic building data are stored in a single data store. Although the BIM-IoTDI method uses the query optimization of the SPARQL engine to execute the static portion of the query, it cannot use this optimization for sensor data because the query is executed on distributed data sources.

A comparison of the BIM-IoTDI method with the GraphDB implementation of the fully semantic integration method is also possible by examining the intervals [1,30000] and (30000,520000] in Figures 9 and 10, respectively. For the interval [1,30000], BIM-IoTDI had a significantly better query response time. However, for the range (30000,520000], the query response time of GraphDB is significantly faster. GraphDB's indexing mechanisms, such as predicate-object-subject (POS) and predicate-subject-object (PSO), are one of the reasons for its faster query response time. Moreover, GraphDB's query optimization contributed to the improved execution time, like the other RDF database engines.

In general, the BIM-IoTDI and fully semantic data integration methods have both advantages and disadvantages. Since this research's primary goal is integrating data for

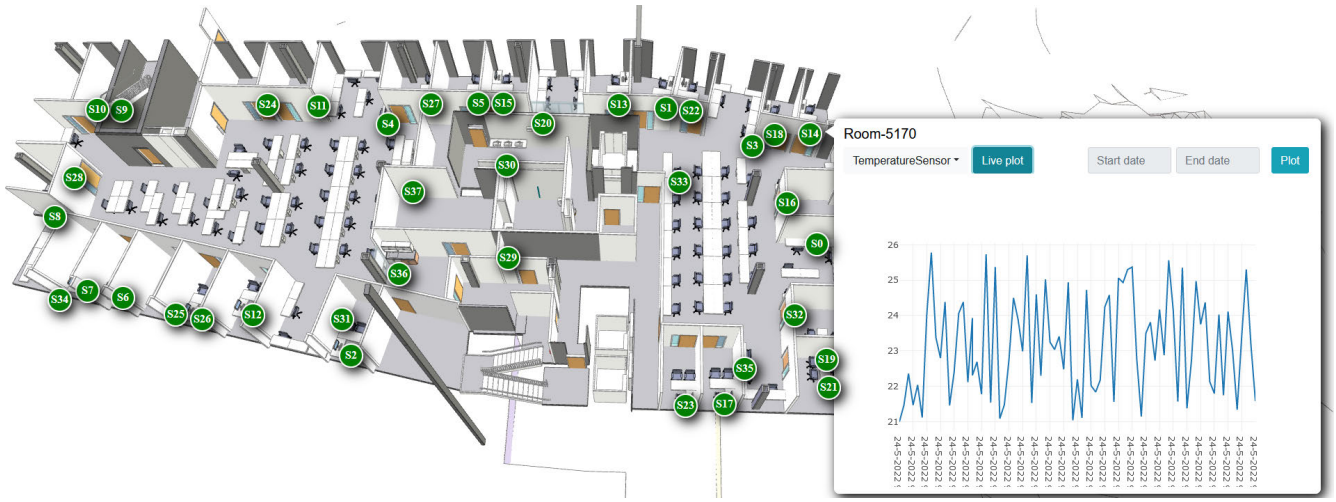


FIGURE 8. Case study real-time monitoring application-displaying real-time sensor readings.

TABLE 4. Property-based comparison of BIM-IoTDI and fully semantic BIM-IoT data integration methods.

Properties	BIM-IoT data integration approach	
	BIM-IoTDI	Fully Semantic
Is semantic interoperability supported?	✓	✓
Does it offer real-time semantic sensor data?	✓	×
Is sensor data retained in an optimal time-series database?	✓	×
Does it prevent sensor data duplication?	✓	×

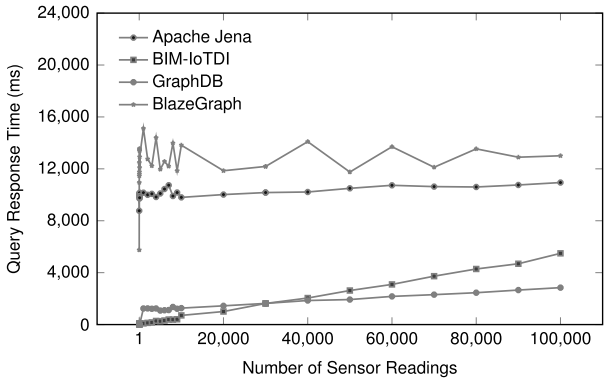


FIGURE 9. Query response time evaluation results (Range: [1,100000]).

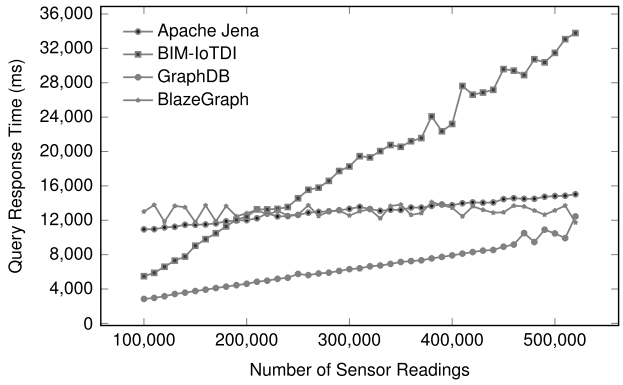


FIGURE 10. Query response time evaluation results (Range: [100000,520000]).

smart-building digital twins, the benefits, drawbacks, and trade-offs are assessed from a smart-building digital twin perspective. Table 4 summarizes a property-based comparison of BIM-IoTDI and fully semantic data integration approaches.

The BIM-IoTDI integration method retains sensor data in its native environment and converts it in real-time to RDF triples. However, the sensor data are initially converted to the RDF format in the fully semantic approach. This conversion is typically not performed in real-time, making it unsuitable for digital twins of smart buildings, where real-time updated data are the primary components. In addition, the initial conversion of sensor data into RDF triples results in data duplication because existing smart building technologies store sensor data in relational or NoSQL databases. Even with a fully semantic approach and real-time conversion of sensor observations into triples, the RDF triples in duplicated RDF sensor data grow exponentially with each sensor observation, resulting in large knowledge graphs quickly. Therefore, due to data duplication and substantial knowledge graph growth, the fully semantic approach becomes ineffective. To explain the extent of data growth with an example scenario, assume six SOSA ontology terms describe each observation (i.e., *sosa:Observation, sosa:hasFeatureOfInterest sosa:hasSimpleResult, sosa:resultTime, sosa:madeBySensor, sosa:ObservedProperty*), and observations are collected half-hourly. In the case study building, there are about three hundred and forty-six sensors according to the BIM model. Therefore, if a fully semantic approach has to be used,

$48 \times 6 \times 346 = 99,648$, $48 \times 6 \times 346 \times 30 = 2,989,440$ $48 \times 6 \times 346 \times 365 = 36,371,520$ triples get generated daily, monthly, and yearly respectively.

Comparing the fully semantic and BIM-IoTDI methods for smart-building digital twins must also consider their applicability in the current smart-building data environment. If a fully semantic integration strategy must be implemented, either all data from various sources must be duplicated into RDF triples, or all existing smart building technologies must support RDF data. However, adopting semantic web technologies in smart building technologies will take time, and duplicating all data into RDF is not an efficient solution. In contrast, BIM-IoTDI acknowledges the independence of data sources and functions by mediating queries instead of storing all data in a central database.

VI. CONCLUSION AND FUTURE WORK

This study introduced a novel multi-layer architecture and a comprehensive framework known as BIM-IoTDI for smart-building digital twins, with a primary focus on enabling semantic interoperability among smart-building digital twin applications. The BIM-IoTDI architecture includes physical, data storage, data access, data integration, and application layers, each of which performs distinct functions. Based on the BIM-IoTDI architecture, a comprehensive framework is presented. The BIM-IoTDI framework outlines the entire process of creating a digital twin of a smart building, from data collection to storage, access, integration, and application development. As the central component of the BIM-IoTDI framework for smart-building digital twins, a data integration method based on domain ontologies and query mediation is developed to provide integrated BIM and IoT data access. The BIM-IoTDI integration method provides semantic static (BIM) and dynamic (IoT) building data that satisfy the real-time data requirements of smart-building digital twins while preserving IoT data in its optimal time-series data storage. In order to validate the feasibility of the BIM-IoTDI smart-building digital twin architecture and framework, an experimental evaluation model is developed using actual building data. A comparison of the BIM-IoTDI and the fully semantic data integration method is then conducted using the experimental evaluation model.

The evaluation results showed that the BIM-IoTDI architecture and framework provide better semantically-interoperable BIM and IoT data and meet the need for real-time updated data in smart-building digital twins. According to the comparative evaluation results, the BIM-IoTDI integration method has a slower query response time for accessing a large number of semantically described sensor observations simultaneously. The relatively slower query response time is primarily the result of the query mediation overhead incurred during the serial transformation of the raw sensor data to an in-memory knowledge graph.

In future work, the BIM-IoTDI architecture will be enhanced by including autonomous decision-making using the substantial domain knowledge embedded within the knowledge graphs produced by this research and machine learning techniques. In addition, the translation of raw sensor data to an in-memory knowledge graph will be parallelized to improve the query response time of the BIM-IoTDI integration approach when accessing a massive amount of data simultaneously. Future research should also focus on the data-sharing aspects of smart-building digital twins.

ACKNOWLEDGMENT

The authors would like to thank the Facilities Management Department, Western University, for providing the data used in this study. In addition, they would also like to thank Dr. Luisa Liboni for her contribution to editing the first draft of this article.

REFERENCES

[1] *2019 Global Status Report for Buildings and Construction: Towards a Zero-Emission, Efficient and Resilient Buildings and Construction Sector*, UNEP GABC, IEA, United Nations Environ. Programme, Nairobi, Kenya, 2019.
[2] *IEA (2020), Tracking Buildings 2020, IEA Paris*. Accessed: Jul. 15, 2022. [Online]. Available: https://www.iea.org/reports/tracking-buildings-2020
[3] S. Kaewunruen, P. Rungskunroch, and J. Welsh, "A digital-twin evaluation of net zero energy building for existing buildings," *Sustainability*, vol. 11, no. 1, p. 159, 2018.
[4] M. Jia, A. Komeily, Y. Wang, and R. S. Srinivasan, "Adopting Internet of Things for the development of smart buildings: A review of enabling technologies and applications," *Automat. Construct.*, vol. 101, pp. 111–126, May 2019.
[5] M. Froufe, C. Chinelli, A. Guedes, A. Haddad, A. Hammad, and C. Soares, "Smart buildings: Systems and drivers," *Buildings*, vol. 10, no. 9, p. 153, Sep. 2020.
[6] A. Verma, S. Prakash, V. Srivastava, A. Kumar, and S. C. Mukhopadhyay, "Sensing, controlling, and IoT infrastructure in smart building: A review," *IEEE Sensors J.*, vol. 19, no. 20, pp. 9036–9046, Oct. 2019.
[7] H. Lasi, P. Fettke, H. G. Kemper, T. Feld, and M. Hoffmann, "Industry 4.0," *Bus. Inf. Syst. Eng.*, vol. 6, no. 4, pp. 239–242, 2014.
[8] G. B. Ozturk, "Digital twin research in the AECO-FM industry," *J. Building Eng.*, vol. 40, Aug. 2021, Art. no. 102730.
[9] E. Forcael, I. Ferrari, A. Opazo-Vega, and J. AlbertoPulido-Arcas, "Construction 4.0: A literature review," *Sustainability*, vol. 12, no. 22, p. 9755, 2020.
[10] R. Berger, "Digitization in the construction industry," *Think Act*, vol. 116, pp. 1–16, 2016.
[11] M. Hotový, "Dynamic model of implementation efficiency of building information modelling (BIM) in relation to the complexity of buildings and the level of their safety," in *Proc. MATEC Web Conf.*, vol. 146, p. 01010.
[12] Z. Liu, N. Meyendorf, and N. Mrad, "The role of data fusion in predictive maintenance using digital twin," in *Proc. AIP Conf.*, Apr. 2018, Art. no. 020023.
[13] S. M. E. Sepasgozar, "Differentiating digital twin from digital shadow: Elucidating a paradigm shift to expedite a smart, sustainable built environment," *Buildings*, vol. 11, no. 4, p. 151, Apr. 2021.
[14] I. Yitmen, S. Alizadehsalehi, İ. Akıner, and M. E. Akıner, "An adapted model of cognitive digital twins for building lifecycle management," *Appl. Sci.*, vol. 11, no. 9, p. 4276, May 2021.
[15] A. Fuller, Z. Fan, C. Day, and C. Barlow, "Digital twin: Enabling technologies, challenges and open research," *IEEE Access*, vol. 8, pp. 108952–108971, 2020.
[16] M. Deng, C. C. Menassa, and V. R. Kamat, "From BIM to digital twins: A systematic review of the evolution of intelligent building representations in the AEC-FM industry," *J. Inf. Technol. Construction*, vol. 26, pp. 58–83, Feb. 2021.
[17] Z. Chevallier, B. Finance, and B. C. Boulakia, "A reference architecture for smart building digital twin," in *Proc. SeDiT ESWC*, vol. 2615, 2020, pp. 1–12.
[18] J. M. D. Delgado and L. Oyedele, "Digital twins for the built environment: Learning from conceptual and process models in manufacturing," *Adv. Eng. Informat.*, vol. 49, Aug. 2021, Art. no. 101332.

[19] Q. Lu, A. K. Parlikad, P. Woodall, G. D. Ranasinghe, X. Xie, Z. Liang, E. Konstantinou, J. Heaton, and J. Schooling, "Developing a digital twin at building and city levels: Case study of West Cambridge campus," *J. Manage. Eng.*, vol. 36, no. 3, May 2020, Art. no. 05020004.

[20] C. Boje, A. Guerriero, S. Kubicki, and Y. Rezgui, "Towards a semantic construction digital twin: Directions for future research," *Autom. Construct.*, vol. 114, Jun. 2020, Art. no. 103179.

[21] Y. Peng, M. Zhang, F. Yu, J. Xu, and S. Gao, "Digital twin hospital buildings: An exemplary case study through continuous lifecycle integration," *Adv. Civil Eng.*, vol. 2020, pp. 1–13, Nov. 2020.

[22] S. Tang, D. R. Shelden, C. M. Eastman, P. Pishdad-Bozorgi, and X. Gao, "A review of building information modeling (BIM) and the Internet of Things (IoT) devices integration: Present status and future trends," *Automat. Construct.*, vol. 101, pp. 127–139, May 2019.

[23] S. Azhar, "Building information modeling (BIM): Trends, benefits, risks, and challenges for the AEC industry," *Leadership Manage. Eng.*, vol. 11, no. 3, pp. 241–252, 2011.

[24] S. H. Khajavi, N. H. Motlagh, A. Jaribion, L. C. Werner, and J. Holmstrom, "Digital twin: Vision, benefits, boundaries, and creation for buildings," *IEEE Access*, vol. 7, pp. 147406–147419, 2019.

[25] A. Roxin, W. Abdou, and W. Derigent, "Interoperable digital building twins through communicating materials and semantic BIM," *Social Netw. Comput. Sci.*, vol. 3, no. 1, pp. 1–25, Jan. 2022.

[26] K. H. Veltman, "Syntactic and semantic interoperability: New approaches to knowledge and the semantic web," *New Rev. Inf. Netw.*, vol. 7, no. 1, pp. 159–183, Jan. 2001.

[27] M. Shahinmoghadam, W. Natephra, and A. Motamedi, "BIM- and IoT-based virtual reality tool for real-time thermal comfort assessment in building enclosures," *Building Environ.*, vol. 199, Jul. 2021, Art. no. 107905.

[28] J. Zhao, H. Feng, Q. Chen, and B. Garcia de Soto, "Developing a conceptual framework for the application of digital twin technologies to revamp building operation and maintenance processes," *J. Building Eng.*, vol. 49, May 2022, Art. no. 104028.

[29] Y. Zhao, N. Wang, Z. Liu, and E. Mu, "Construction theory for a building intelligent operation and maintenance system based on digital twins and machine learning," *Buildings*, vol. 12, no. 2, p. 87, Jan. 2022.

[30] L. Zhao, C. Wang, K. Zhao, D. Tarchi, S. Wan, and N. Kumar, "INTERLINK: A digital twin-assisted storage strategy for satellite-terrestrial networks," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 58, no. 5, pp. 3746–3759, Oct. 2022.

[31] G. Desogus, E. Quaquero, G. Rubiu, G. Gatto, and C. Perra, "BIM and IoT sensors integration: A framework for consumption and indoor conditions data monitoring of existing buildings," *Sustainability*, vol. 13, no. 8, p. 4496, Apr. 2021.

[32] C. M. Eastman, C. Eastman, P. Teicholz, R. Sacks, and K. Liston, *BIM Handbook: A Guide to Building Information Modeling for Owners, Managers, Designers, Engineers and Contractors*. Hoboken, NJ, USA: Wiley, 2011.

[33] M. Delavar, G. T. Bitsuamlak, J. K. Dickinson, and L. M. F. Costa, "Automated BIM-based process for wind engineering design collaboration," *Building Simul.*, vol. 13, no. 2, pp. 457–474, Apr. 2020.

[34] *Industry Foundation Classes (IFC) for Data Sharing in the Construction and Facility Management Industries*, Standard ISO 16739:2013, BuildingSMART Internationl, 2013.

[35] P. Pauwels, S. Zhang, and Y.-C. Lee, "Semantic web technologies in AEC industry: A literature overview," *Autom. Construction*, vol. 73, pp. 145–165, Jan. 2017.

[36] S. Madakam, R. Ramaswamy, and S. Tripathi, "Internet of Things (IoT): A literature review," *J. Comput. Commun.*, vol. 3, no. 5, p. 164, 2015.

[37] S. Li, L. Xu, and S. Zhao, "The Internet of Things: A survey," *Inf. Syst. Frontiers*, vol. 17, no. 2, pp. 243–259, 2015.

[38] F. Wortmann and K. Fluchter, "Internet of Things," *Bus. Inf. Syst. Eng.*, vol. 57, no. 3, pp. 221–224, 2015.

[39] C. Chen, J. Jiang, Y. Zhou, N. Lv, X. Liang, and S. Wan, "An edge intelligence empowered flooding process prediction using Internet of Things in smart city," *J. Parallel Distrib. Comput.*, vol. 165, pp. 66–78, Jul. 2022.

[40] J.-H. Woo, M. A. Peterson, and B. Gleason, "Developing a virtual campus model in an interactive game-engine environment for building energy benchmarking," *J. Comput. Civil Eng.*, vol. 30, no. 5, Sep. 2016. Art. no. C4016005.

[41] X. Yuan, C. J. Anumba, and M. K. Parfitt, "Cyber-physical systems for temporary structure monitoring," *Autom. Construct.*, vol. 66, pp. 1–14, Jun. 2016.

[42] T. Gerrish, K. Ruikar, M. Cook, M. Johnson, M. Phillip, and C. Lowry, "BIM application to building energy performance visualisation and management: Challenges and potential," *Energy Buildings*, vol. 144, pp. 218–228, Jun. 2017.

[43] A. Khalili and D. K. H. Chua, "IFC-based graph data model for topological queries on building elements," *J. Comput. Civil Eng.*, vol. 29, no. 3, May 2015, Art. no. 04014046.

[44] W. Solihin, C. Eastman, Y.-C. Lee, and D.-H. Yang, "A simplified relational database schema for transformation of BIM data into a query-efficient and spatially enabled database," *Autom. Construction*, vol. 84, pp. 367–383, Dec. 2017.

[45] A. Motamedi, A. Hammad, and Y. Asen, "Knowledge-assisted BIM-based visual analytics for failure root cause detection in facilities management," *Autom. Construct.*, vol. 43, pp. 73–83, Jul. 2014.

[46] T.-W. Kang and H.-S. Choi, "BIM perspective definition metadata for interworking facility management data," *Adv. Eng. Informat.*, vol. 29, no. 4, pp. 958–970, Oct. 2015.

[47] M. Dibley, H. Li, Y. Rezgui, and J. Miles, "An ontology framework for intelligent sensor-based building monitoring," *Autom. Construct.*, vol. 28, pp. 1–14, Dec. 2012.

[48] E. Curry, J. O'Donnell, E. Corry, S. Hasan, M. Keane, and S. O'Riain, "Linking building data in the cloud: Integrating cross-domain building data using linked data," *Adv. Eng. Informat.*, vol. 27, no. 2, pp. 206–219, Apr. 2013.

[49] T. Berners-Lee, J. Hendler, and O. Lassila, "The semantic web," *Sci. Amer.*, vol. 284, no. 5, pp. 34–43, May 2001.

[50] A. G. Mohamed, M. R. Abdallah, and M. Marzouk, "BIM and semantic web-based maintenance information for existing buildings," *Autom. Construct.*, vol. 116, Aug. 2020, Art. no. 103209.

[51] S. Hu, E. Corry, E. Curry, W. J. N. Turner, and J. O'Donnell, "Building performance optimisation: A hybrid architecture for the integration of contextual information and time-series data," *Autom. Construction*, vol. 70, pp. 51–61, Oct. 2016.

[52] K. McGlinn, B. Yuce, H. Wicaksono, S. Howell, and Y. Rezgui, "Usability evaluation of a web-based tool for supporting holistic building energy management," *Autom. Construct.*, vol. 84, pp. 154–165, Dec. 2017.

[53] J. J. Hunhevicz, M. Motie, and D. M. Hall, "Digital building twins and blockchain for performance-based (smart) contracts," *Autom. Construct.*, vol. 133, Jan. 2022, Art. no. 103981.

[54] A. Schweigkofler, O. Braholli, S. Akro, D. Siegele, P. Penna, C. Marcher, L. Tagliabue, and D. Matt, "Digital Twin as energy management tool through IoT and BIM data integration," in *Proc. CLIMA Conf.*, 2022.

[55] A. Zaballos, A. Briones, A. Massa, P. Centelles, and V. Caballero, "A smart campus' digital twin for sustainable comfort monitoring," *Sustainability*, vol. 12, pp. 1–33, Nov. 2020.

[56] Q. Lu, X. Xie, A. K. Parlikad, and J. M. Schooling, "Digital twin-enabled anomaly detection for built asset monitoring in operation and maintenance," *Autom. Construct.*, vol. 118, Oct. 2020, Art. no. 103277.

[57] L. Chamari, E. Petrova, and P. Pauwels, "A web-based approach to BMS, BIM and IoT integration: A case study," in *Proc. CLIMA Conf.*, 2022, pp. 2628–2635.

[58] R. Cyganiak, D. Wood, M. Lanthaler, G. Klyne, J. J. Carroll, and A. B. McBride, "RDF 1.1 concepts and abstract syntax," *W3C recommendation*, vol. 25, no. 2, pp. 1–22, 2014.

[59] M. Friedemann, K. Wenzel, and A. Singer, "Linked data architecture for assistance and traceability in smart manufacturing," in *Proc. MATEC Web Conf.*, vol. 304, 2019, p. 04006.

[60] N. Moretti, X. Xie, J. Merino, J. Brazauskas, and A. K. Parlikad, "An openBIM approach to IoT integration with incomplete as-built data," *Appl. Sci.*, vol. 10, no. 22, p. 8287, Nov. 2020.

[61] B. Balaji, A. Bhattacharya, G. Fierro, J. Gao, J. Gluck, D. Hong, A. Johansen, J. Koh, J. Ploennigs, Y. Agarwal, M. Bergés, D. Culler, R. K. Gupta, M. B. Kjærgaard, M. Srivastava, and K. Whitehouse, "Brick : Metadata schema for portable smart building applications," *Appl. Energy*, vol. 226, pp. 1273–1292, Sep. 2018.

[62] G. A. Benndorf, D. Wystrcil, and N. Réhault, "Energy performance optimization in buildings: A review on semantic interoperability, fault detection, and predictive control," *Appl. Phys. Rev.*, vol. 5, no. 4, Dec. 2018, Art. no. 041501.

[63] P. Pauwels and W. Terkaj, "Express to owl for construction industry: Towards a recommendable and usable ifcOWL ontology," *Autom. Construct.*, vol. 63, pp. 100–133, Mar. 2016.

[64] M. H. Rasmussen, M. Lefrançois, G. F. Schneider, and P. Pauwels, "BOT: The building topology ontology of the W3C linked building data group," *Semantic Web*, vol. 12, no. 1, pp. 143–161, Nov. 2020.

[65] A. Haller, K. Janowicz, S. J. Cox, M. Lefrançois, K. Taylor, D. Le Phuoc, J. Lieberman, R. García-Castro, R. Atkinson, and C. Stadler, "Th SOSA/SSN ontology: A joint W3C and OGC standard specifying the semantics of sensors, observations, actuation, and sampling," *Semantic Web*, vol. 1, pp. 1–19, Jan. 2018.
[66] B. Balaji, A. Bhattacharya, G. Fierro, J. Gao, J. Gluck, D. Hong, A. Johansen, J. Koh, J. Ploennigs, Y. Agarwal, M. Berges, D. Culler, R. Gupta, M. B. Kjelgaard, M. Srivastava, and K. Whitehouse, "Brick: Towards a unified metadata schema for buildings," in *Proc. 3rd ACM Int. Conf. Syst. Energy-Efficient Built Environ.*, Nov. 2016, pp. 41–50.
[67] R. Hodgson, P. J. Keller, J. Hodges, and J. Spivak. (Mar. 2014). *QUDT-Quantities, Units, Dimensions and Data Types Ontologies*. [Online]. Available: http://qudt.org
[68] *SPARQL 1.1 Overview*, World Wide Web Consortium, Cambridge, MA, USA, 2013.
[69] M. Mosser, F. Pieressa, J. Reutter, A. Soto, and D. Vrgoč, "Querying APIs with SPARQL: Language and worst-case optimal algorithms," in *Proc. Eur. Semantic Web Conf.*, vol. 10843. Heraklion, Greece: Springer, 2018, pp. 639–654.
[70] M. H. Rasmussen, "Madsholten/revit-BOT-exporter export from autodesk revit to BOT-compliant turtle," Tech. Rep., 2018. Accessed: Feb. 13, 2022. [Online]. Available: https://github.com/MadsHolten/revit-bot-exporter

DAGIMAWI D. ENEYEW (Graduate Student Member, IEEE) received the B.Sc. and M.Sc. degrees in electrical engineering, computer engineering from Addis Ababa University, Addis Ababa, Ethiopia, in 2015 and 2017, respectively. He is currently pursuing the Ph.D. degree in electrical and computer engineering, software program with Western University, Canada. He worked as a Lecturer at the School of Electrical and Computer Engineering, Addis Ababa University Institute of Technology, Addis Ababa, from 2015 to 2019. His research interests include digital twins, big data, machine learning, and smart buildings.

MIRIAM A. M. CAPRETZ (Senior Member, IEEE) received the B.Sc. and M.E.Sc. degrees from the Universidade Estadual de Campinas (UNICAMP), Brazil, and the Ph.D. degree from the University of Durham, U.K. She was with the University of Aizu, Japan. She is currently a Professor with the Department of Electrical and Computer Engineering, Western University, Canada. She has been involved in the software engineering area for more than 35 years. She has also involved with the organization of workshops and symposia and has been serving on program committees at international conferences. Her current research interests include digital twins, cloud computing, big data, machine learning, and service-oriented architecture.

GIRMA T. BITSUAMLAK received the B.Sc. degree from Addis Ababa University, Ethiopia, the M.Tech. degree from IIT Roorkee, India, and the Ph.D. degree from Concordia University, Canada. He is currently a Professor with the Department of Civil and Environmental Engineering, Western University, Canada. He is also the Director of the Wind Energy Environment Engineering (WindEEE) Research Facilities, Western University. He serves as the Western's Site-Leader for the Sharcnet Advanced Research Computing Centre. His current research interests include sustainable and climate-resilient buildings, climate change, machine learning, computational fluid dynamics, digital twin, and wind tunnels. He is a fellow of the Canadian Society of Civil Engineers.